UNIWERSYTET MARII CURIE-SKŁODOWSKIEJ
W LUBLINIE
Wydział Matematyki, Fizyki i Informatyki

Kierunek: **Geoinformatyka**
Specjalność:

Krystian Banach
Nr albumu: 307619

Projekt i implementacja oprogramowania do analizy i przetwarzania tras GPX na potrzeby zawodów sportowych

Design and implementation of software for analyzing and processing GPX routes for sports competitions

Praca magisterska
napisana w Katedrze Oprogramowania Systemów Informatycznych
Instytutu Informatyki
pod kierunkiem dr Michała Klisowskiego

Lublin 2026

Spis treści

Wstęp **5**

1 Teoria **7**

1.1 Pamięć 7

1.1.1 Pamięć podręczna 7

1.1.2 Równoległe wykonanie operacji 9

1.1.3 Dane GPS/GNSS oraz formaty danych 10

Wstęp

kiedyś tu coś napiszę

Rozdział 1

Teoria

1.1 Pamięć

1.1.1 Pamięć podręczna

Współczesne systemy komputerowe są projektowane w oparciu o hierarchię pamięci: jednostka centralna wykonuje instrukcje znacznie szybciej, niż możliwa jest komunikacja z pamięcią operacyjną, dlatego pomiędzy CPU a RAM stosuje się jeden lub kilka poziomów pamięci podręcznej. Istotą cache jest ograniczenie liczby kosztownych odwołań do pamięci wolniejszej poprzez przechowywanie danych używanych często lub ostatnio, co zmniejsza oczekiwanie procesora na dane.[1] W materiałach dotyczących architektury komputerów podkreśla się, że hierarchia obejmuje co najmniej rejestry, poziomy cache oraz pamięć operacyjną, a różnice czasów dostępu pomiędzy tymi warstwami są na tyle duże, że ignorowanie ich w analizie wydajności prowadzi do błędnych wniosków.[2]

Skuteczność działania pamięci podręcznej jest ściśle powiązana z tzw. zasadą lokalności odwołań. Lokalność czasowa oznacza, że dane użyte w niedalekiej przeszłości mają zwiększone prawdopodobieństwo ponownego użycia, natomiast lokalność przestrzenna – że odwołania do adresów sąsiadujących w przestrzeni adresowej pojawiają się blisko siebie w czasie.[2] W kontekście formalnym zachowanie to bywa opisywane m.in. modelem zbioru roboczego (working set), w którym zakłada się, że w pewnym oknie czasowym program intensywnie operuje na ograniczonym podzbiorze stron pamięci; koncepcja ta stanowi jedno z klasycznych wyjaśnień, dlaczego dobry wzorzec odwołań do pamięci może zasadniczo poprawiać wydajność.[3] W praktyce oznacza to, że programy przetwarzające dane sekwencyjne (np. kolejne próbki trasy) mają naturalną tendencję do korzystnego wykorzystania cache, o ile implementacja nie wprowadza niepotrzebnego rozproszenia danych.

Z punktu widzenia implementacji kluczowe jest, że pamięć podręczna nie przenosi pojedynczych bajtów, lecz operuje na blokach (wierszach/liniach cache) o ustalonym

rozmiarze zależnym od architektury (w dydaktycznych opisach spotyka się przykładowe wartości rzędu kilkunastu–kilkudziesięciu bajtów i więcej). Skutkiem tego pojedynczy odczyt może spowodować załadowanie do cache całego sąsiedztwa odczytywanego adresu, co wzmacnia korzyści z lokalności przestrzennej.[1] Chybienie w cache (cache miss) skutkuje koniecznością pobrania danych z niższego poziomu hierarchii (kolejnego cache lub RAM), co jest znacząco droższe czasowo od trafienia (cache hit) i może dominować w czasie wykonania fragmentu programu.[2] Zjawiska te są szczególnie istotne w obliczeniach na długich sekwencjach, gdzie nawet prosta pętla może być ograniczona przepustowością i opóźnieniami dostępu do pamięci.

Konsekwencją powyższego jest to, że dobór struktur danych należy rozpatrywać nie tylko w kategoriach wygody programistycznej, lecz również w kategoriach ułożenia danych w pamięci. Struktury o ciągłym układzie (np. tablice) sprzyjają przetwarzaniu sekwencyjnemu, ponieważ kolejne elementy są fizycznie sąsiadujące, a więc jedna operacja pobrania bloku danych zwiększa prawdopodobieństwo, że następne odwołania również będą obsłużone z cache. Struktury rozproszone (np. obiekty powiązane referencjami/wskaźnikami, z wieloma niezależnymi alokacjami) prowadzą częściej do skoków po przestrzeni adresowej i zwiększają ryzyko seryjnych chybień w cache. Z perspektywy projektowania systemu do analizy tras GPX oznacza to, że reprezentacja danych powinna minimalizować nieciągłości pamięciowe wszędzie tam, gdzie wykonywane są masowe operacje „punkt po punkcie”.

W literaturze poświęconej analizie wydajności podkreśla się rozróżnienie pomiędzy kodem ograniczonym możliwościami obliczeniowymi a kodem ograniczonym szybkością pamięci. Model Roofline porządkuje to rozumowanie, wskazując, że dla niskiej intensywności obliczeń (mało operacji na bajt danych) czynnikiem limitującym staje się przepustowość pamięci, a nie „moc” jednostek arytmetycznych.[4] W analizie tras sportowych wiele operacji ma właśnie taki charakter: na każdą próbkę wykonuje się relatywnie niewiele działań arytmetycznych, natomiast trzeba odczytać (i często zapisać) sporo danych pośrednich. W takich warunkach optymalizacja dostępu do pamięci (a więc m.in. wybór struktur danych wspierających lokalność) bywa bardziej opłacalna niż mikrooptymalizacje samych wzorów obliczeniowych.

Znaczenie lokalności dobrze ilustrują przykłady z dydaktyki przetwarzania równoległego i optymalizacji pętli: kolejność iteracji w pętlach oraz sposób ułożenia tablic mogą zwiększać lub zmniejszać lokalność przestrzenną i czasową, a przez to istotnie wpływać na liczbę chybień do cache i koszty translacji adresów (TLB). W materiałach laboratoryjnych dotyczących analizy efektywności mnożenia macierzy wprost wiąże się lokalność z kolejnością pętli i odległością elementów w pamięci.[5] Podobny efekt, przedstawiony w formie popularnonaukowej, pokazuje przykład zamiany kolejności pętli w programie operującym na tablicy, gdzie sama zmiana porządku dostępu do danych może dawać

wielokrotne różnice czasu działania, mimo identycznej liczby operacji arytmetycznych.[6] Wniosek praktyczny pozostaje ten sam: procesor wykonuje szybciej operacje, tylko wtedy, gdy dane są podawane w sposób zgodny z mechaniką cache.

1.1.2 Równoległe wykonanie operacji

Współczesne procesory osiągają wysoką wydajność nie tylko dzięki wzrostowi częstotliwości taktowania, lecz przede wszystkim poprzez rozwijanie różnych form równoległości. Klasycznym sposobem uporządkowania tych form jest taksonomia Flynna, która klasyfikuje architektury według liczby strumieni instrukcji oraz liczby strumieni danych. W najczęściej przytaczanym ujęciu wyróżnia się architektury SISD, SIMD, MISD oraz MIMD, gdzie w praktyce obliczeń inżynierskich i analizy danych dominują przypadki SIMD i MIMD.Taksonomia ta stanowi użyteczny sposób opisu współczesnych platform obliczeniowych, ponieważ pozwala rozdzielić równoległość na poziomie danych (SIMD) od równoległości na poziomie zadań i wątków (MIMD).[7]

W kontekście analizy tras GPX szczególnie istotna jest równoległość typu SIMD (Single Instruction, Multiple Data), w której ta sama operacja wykonywana jest jednocześnie na wielu elementach danych. Zjawisko to jest bezpośrednio powiązane z wektoryzacją, czyli przekształcaniem pętli i wyrażeń tak, aby możliwe było użycie instrukcji wektorowych oraz rejestrów o zwiększonej szerokości (np. rodziny instrukcji AVX w architekturze x86). W praktyce przekłada się to na sytuację, w której pojedyncza instrukcja arytmetyczna może obsłużyć kilka wartości zmiennoprzecinkowych w jednym kroku wykonania, o ile dane są ułożone w pamięci w sposób sprzyjający przetwarzaniu blokowemu oraz nie występują zależności uniemożliwiające równoległe wykonanie iteracji.[8]

Wektoryzacja nie stanowi jednak automatycznej korzyści z zastosowania szybkiego procesora. W dokumentacji optymalizacyjnej producentów procesorów podkreśla się, że na skuteczność SIMD wpływają m.in.: regularność pętli, unikanie trudnych do przewidzenia rozgałęzień, ograniczanie aliasowania (nakładania się obszarów pamięci) oraz zapewnienie korzystnych wzorców dostępu do danych.[8] W praktyce oznacza to, że obliczenia typowe dla tras (np. różnice między kolejnymi próbkami, obliczanie odległości odcinków, filtrowanie sygnałów) są dobrymi kandydatami do wektoryzacji wtedy, gdy realizuje się je na możliwie prostych, jednorodnych tablicach danych, a nie na rozproszonych strukturach obiektowych.[8]

W standardowych modelach programowania równoległego spotyka się również mechanizmy deklaratywne wspierające wektoryzację. Przykładowo, w specyfikacji OpenMP przewidziano konstrukcję simd, której znaczenie polega na wskazaniu kompilatorowi, że iteracje danej pętli mogą być wykonywane współbieżnie z użyciem instrukcji SIMD (o ile spełnione są wymagania poprawnościowe).[9] Z perspektywy projektowania oprogramowania do analizy tras istotne jest nie tyle samo użycie dyrektyw, co wynikająca z nich

zasada: należy konstruować krytyczne wydajnościowo fragmenty obliczeń w postaci prostych pętli nad uporządkowanymi danymi, aby umożliwić kompilatorowi (lub bibliotece numerycznej) wykonanie wektoryzacji.[9]

Drugą kluczową formą równoległości jest MIMD (Multiple Instruction, Multiple Data), czyli wykonywanie różnych strumieni instrukcji na różnych danych, co odpowiada m.in. przetwarzaniu wielowątkowemu na CPU wielordzeniowym. W przypadku zastosowań sportowych naturalnym scenariuszem MIMD jest równoległe przetwarzanie wielu tras (np. wielu zawodników) lub wielu plików, gdzie poszczególne zadania mogą być wykonywane niezależnie. Jednocześnie należy uwzględnić ograniczenia teoretyczne i praktyczne: prawo Amdahla wskazuje, że maksymalne przyspieszenie jest ograniczone częścią sekwencyjną programu, nawet przy zwiększaniu liczby procesorów/wątków.[10] Zjawisko to ma znaczenie zwłaszcza wtedy, gdy w potoku przetwarzania występuje etap trudny do zrównoleglenia (np. pewne formy parsowania, walidacji, agregacji wyników) lub gdy głównym ograniczeniem staje się przepustowość pamięci, a nie liczba rdzeni.[10]

Klasyfikacja MIMD dla systemów równoległych jest często rozwijana o kwestie związane ze sposobem organizacji pamięci i komunikacji (np. podział na systemy z pamięcią globalną lub rozproszoną oraz z komunikacją przez zmienne współdzielone lub przesyłanie komunikatów). Tego rodzaju ujęcie porządkuje dobór modelu programowania do docelowej platformy uruchomieniowej (komputer osobisty, serwer wieloprocesorowy, klaster), co bywa istotne w projektach, w których przetwarzanie dużej liczby tras ma charakter masowy.[11] Przekłada się to na fakt, że równoległość MIMD może być realizowana zarówno w obrębie jednego węzła (wątki/procesy), jak i w rozwiązaniach skalowanych w modelu rozproszonym, o ile architektura aplikacji zostanie do tego przygotowana.[11]

1.1.3 Dane GPS/GNSS oraz formaty danych

Dane pozycyjne wykorzystywane w rejestracji aktywności sportowych mają najczęściej charakter sekwencyjny: stanowią uporządkowany w czasie zbiór próbek opisujących pozycję (szerokość i długość geograficzną), często także wysokość oraz znacznik czasu. Należy zauważyć, że w praktyce pomiar GNSS/GPS jest obarczony wieloma źródłami błędów, które mogą istotnie wpływać na analizę pochodnych wielkości (np. prędkości chwilowej, przewyższeń). Wśród typowych przyczyn odchyleń wskazuje się m.in. wielotorowość (multipath), czyli docieranie sygnału do odbiornika wieloma drogami w wyniku odbić, co skutkuje zaburzeniem estymaty odległości i pozycji.[12] Konsekwencją projektową jest konieczność stosowania metod, które są odporne na szum i artefakty: filtracji, wygładzania, wykrywania odstających próbek oraz ostrożnego różniczkowania sygnałów (np. różnic czasu i odległości) w celu wyznaczania prędkości.[12]

Zapis pozycji w urządzeniach konsumenckich i formatach wymiany opiera się zazwyczaj o globalny układ odniesienia WGS84, w którym pozycja opisywana jest współrzęd-

nymi geograficznymi (lat, lon) odnoszonymi do elipsoidy odniesienia.[13] W przypadku GPX jest to sformułowane wprost w dokumentacji schematu: współrzędne są deklarowane jako względne względem WGS84, a pomiary przyjmują jednostki metryczne.[14] Dla przetwarzania oznacza to, że dane wejściowe są zwykle w stopniach (dla szerokości i długości), co wymaga konsekwentnego doboru metod obliczeniowych (np. odpowiednich wzorów na odległość po powierzchni Ziemi) oraz jawnego zarządzania jednostkami i konwersjami.[13][14]

W ramach formatów plikowych szczególne znaczenie ma GPX, czyli format wymiany oparty o XML. Dokumentacja GPX 1.1 opisuje typową strukturę pliku jako metadane oraz zbiory punktów (waypoints), tras (routes) i śladów (tracks), gdzie ślad (trk) może zawierać segmenty (trkseg), a segmenty – kolejne punkty śladu (trkpt).[14] Pola takie jak czas (time) i wysokość (ele) występują jako elementy opcjonalne, co w praktyce wymaga od implementacji odporności na brak danych oraz obsługi różnych wariantów zapisu generowanych przez odmienne urządzenia i aplikacje.[14] Należy również pamiętać, że choć format GPX zapewnia czytelność i interoperacyjność, to jako plik XML generuje narzut parsowania oraz zajmuje więcej miejsca w pamięci w porównaniu z formatami binarnymi, co staje się istotne podczas masowego przetwarzania danych.[14]

Z perspektywy projektu oprogramowania do analizy tras GPX należy zatem rozdzielić dwie warstwy: (1) warstwę wejścia/wyjścia, w której niezbędne jest poprawne parsowanie i walidacja struktury XML, oraz (2) warstwę obliczeń, w której kluczowe jest przekształcenie danych do reprezentacji sprzyjającej przetwarzaniu masowemu. W praktyce oznacza to konwersję listy punktów śladu do postaci tablicowej (np. osobne wektory lat, lon, ele, czasu), co pozwala wykonywać obliczenia strumieniowo, a także zwiększa szansę na wykorzystanie mechanizmów cache i wektoryzacji.[8][14]

Bibliografia

[1] T. Kapeła, *Architektura systemów informatycznych. Architektura i organizacja pamięci*, materiał dydaktyczny (PDF), tryb dostępu: `https://ww2.ii.uj.edu.pl/~kapela/pn/ASI5.pdf`, (dostęp: 2 marca 2026 r.).

[2] J. Kazimirski, *Architektura komputerów*, materiał dydaktyczny (PDF), tryb dostępu: `https://jankazimirski.pl/files/5AKW.07.pdf`, (dostęp: 2 marca 2026 r.).

[3] P. J. Denning, *The Working Set Model for Program Behavior*, „Communications of the ACM" 1968, t. 11, nr 5, s. 323–333, tryb dostępu: `https://denninginstitute.com/pjd/PUBS/WSModel_1968.pdf`, (dostęp: 2 marca 2026 r.).

[4] Platforma PLGrid, *Model Roofline*, tryb dostępu: `https://kompendium.plgrid.pl/podstawy/pojecia/roofline/`, (dostęp: 2 marca 2026 r.).

[5] R. Walkowiak, *Materiały pomocnicze do laboratorium z przetwarzania równoległego (maj 2019). Analiza efektywności mnożenia macierzy w systemach z pamięcią współdzieloną*, materiał dydaktyczny (PDF), tryb dostępu: `https://www.cs.put.poznan.pl/rwalkowiak/pliki/2019/pr/MM_2019.pdf`, (dostęp: 2 marca 2026 r.).

[6] K. Piecuch, *Pamięć cache w praktyce*, „Delta" 2009, nr 10, tryb dostępu: `https://www.deltami.edu.pl/media/articles/2009/10/delta-2009-10-pamiec-cache-w-praktyce.pdf`, (dostęp: 2 marca 2026 r.).

[7] L. Szustak, *Architektura i elementy składowe systemów komputerowych* (wykład; slajdy), Politechnika Częstochowska, (fragment dot. taksonomii Flynna), tryb dostępu: `https://icis.pcz.pl/~lszustak/students/ASK/wyklady/ASK_L03.pdf`, (dostęp: 2 marca 2026 r.).

[8] Intel Corporation, *Intel® 64 and IA-32 Architectures Optimization Reference Manual*, (rozdziały dot. optymalizacji pętli i SIMD), tryb dostępu: `https://cdrdv2-public.intel.com/814198/248966-Optimization-Reference-Manual-V1-049.pdf`, (dostęp: 2 marca 2026 r.).

[9] OpenMP Architecture Review Board, *OpenMP API Specification 5.1 – simd construct*, tryb dostępu: `https://www.openmp.org/spec-html/5.1/openmpsu49.html`, (dostęp: 2 marca 2026 r.).

[10] R. Walkowiak, *Analiza efektywności przetwarzania współbieżnego* (materiały dydaktyczne), Politechnika Poznańska, (fragment dot. prawa Amdahla), tryb dostępu: `https://www.cs.put.poznan.pl/rwalkowiak/pliki/2016/pr/pr_efektywnosc_2014.pdf`, (dostęp: 2 marca 2026 r.).

[11] T. Zięba, *Przetwarzanie równoległe w rozwiązywaniu problemu dopasowania struktur molekularnych* (rozprawa doktorska), (fragment o rozszerzeniu klasyfikacji MIMD), tryb dostępu: `https://zbc.uz.zgora.pl/repozytorium/Content/17006/HTML/doktorat_TZieba.pdf`, (dostęp: 2 marca 2026 r.).

[12] J. Januszewski, *Systemy satelitarne GPS, Galileo i inne*, Wydawnictwo Naukowe PWN, Warszawa, 2010.

[13] *Obowiązujące układy współrzędnych a lokalizacja przestrzenna* (materiał informacyjny), Uniwersytet Morski w Gdyni, (fragment o WGS84), tryb dostępu: `https://www.umgdy.gov.pl/wp-content/uploads/2017/01/zg-mgm-uklady-wspolrzednych.pdf`, (dostęp: 2 marca 2026 r.).

[14] Topografix, *GPX 1.1 Schema Documentation*, (informacje o strukturze GPX i odniesieniu do WGS84), tryb dostępu: `https://www.topografix.com/gpx/1/1/`, (dostęp: 2 marca 2026 r.).